

PRÉSENTATION

DESCRIPTION : Les builtins sont directement disponibles sans import.
Python 3.12 en compte 71 fonctions/types et quelques constantes.

POURQUOI : Connaître les builtins évite de réinventer la roue et d'importer inutilement des modules externes.

CONTEXTE : Tout script Python, manipulation de données, algorithmes.

CATÉGORIES :

Entrée/Sortie	print, input, open
Conversions de type	int, float, str, bool, bytes, bytearray, complex
Types conteneurs	list, tuple, dict, set, frozenset
Numériques	abs, round, divmod, pow, sum, min, max
Itérables	len, range, enumerate, zip, map, filter, reversed, sorted
Fonctionnelles	all, any, next, iter, callable
Inspection	type, isinstance, issubclass, id, hash, dir, vars, repr
Objets/Classes	object, super, property, classmethod, staticmethod
Attributs	getattr, setattr, delattr, hasattr
Évaluation	eval, exec, compile, __import__
Mémoire/Système	memoryview, slice, format, chr, ord

ACCÈS :

dir(__builtins__)	Liste tous les builtins
import builtins; dir(builtins)	Identique
help(len)	Documentation d'un builtin

Entrée / Sortie

DESCRIPTION : Interaction avec l'utilisateur et les fichiers.

POURQUOI : Communication avec le monde extérieur.

CONTEXTE : Scripts interactifs, logging, lecture de fichiers.

print() :

print()	Affiche une ligne vide
print("hello")	→ hello
print("a", "b", "c")	→ a b c
print("a", "b", sep=", ")	→ a, b
print("a", "b", sep="")	→ ab
print("hello", end="")	Sans saut de ligne
print("hello", end="\n\n")	Deux sauts
print("a", "b", file=sys.stderr)	Vers stderr
print(*liste)	Dépaquète la liste
print(f"x={x:.2f}")	f-string
print(repr(x))	Représentation officielle
print("-" * 40)	Séparateur

input() :

x = input()	Lit une ligne (sans prompt)
x = input("Entrez votre nom : ")	Avec prompt
n = int(input("Nombre : "))	Convertit directement
# input() retourne toujours une str	
# input() lève EOFError si stdin est fermé (ex. pipe vide)	
# Ctrl+C → KeyboardInterrupt, Ctrl+D → EOFError	

open() :

open(file, mode="r")	Ouvre un fichier
open("f.txt", "r")	Lecture texte
open("f.txt", "w")	Écriture texte (écrase)
open("f.txt", "a")	Ajout en fin
open("f.txt", "x")	Création exclusive (FileExistsError si existant)
open("f.txt", "r+")	Lecture + écriture
open("f.bin", "rb")	Lecture binaire
open("f.bin", "wb")	Écriture binaire
open("f.bin", "rb+")	Lecture + écriture binaire
open("f.txt", encoding="utf-8")	Encodage explicite
open("f.txt", encoding="utf-8", errors="replace")	
open("f.txt", errors="ignore")	
open("f.txt", newline="")	Pas de normalisation des fins de ligne

open("f.txt", buffering=-1)	Bufferisation par défaut
open("f.txt", buffering=0)	Pas de buffer (binaire seulement)
open("f.txt", buffering=1)	Ligne par ligne
open("f.txt", buffering=8192)	Buffer personnalisé
with open("f.txt") as f:	Gestionnaire de contexte (recommandé)
contenu = f.read()	
with open("f.txt", "w") as f:	
f.write("hello\n")	
# Méthodes des fichiers texte :	
f.read()	Tout le contenu → str
f.read(n)	n caractères
f.readline()	Une ligne (avec \n)
f.readlines()	Toutes les lignes → liste
f.write(s)	Écrit s, retourne len(s)
f.writelines(lignes)	Écrit une liste de chaînes
f.seek(0)	Retourne au début
f.seek(n)	Positionne à l'octet n
f.tell()	Position courante
f.flush()	Vide le buffer
f.close()	Ferme (automatique avec with)
f.closed	True si fermé
f.name	Nom du fichier
f.mode	Mode d'ouverture
for ligne in f:	Itère ligne par ligne (efficace)

Conversions numériques

DESCRIPTION : Conversion et création de types numériques.
 POURQUOI : Typier correctement les données, changer de base, valider.
 CONTEXTE : Parsing de données, calculs, interfaces utilisateur.

int() :	
int()	→ 0
int(3.7)	→ 3 (troncature vers 0)
int(-3.7)	→ -3 (troncature, pas floor !)
int("42")	→ 42
int(" 42 ")	→ 42 (ignore les espaces)
int("-17")	→ -17
int("0xFF", 16)	→ 255 (base 16)
int("0b1010", 2)	→ 10 (base 2, avec préfixe)
int("1010", 2)	→ 10 (base 2, sans préfixe)
int("0o17", 8)	→ 15 (base 8)
int("FF", 16)	→ 255 (hexadécimal)
int("z", 36)	→ 35 (base 36)
int(True)	→ 1
int(False)	→ 0
int("3.14")	→ ValueError (pas de float en str)
int(3.14e10)	→ 31400000000
# int en Python = entier de précision arbitraire (pas d'overflow)	
int(10**100)	→ googol (exact !)
float() :	
float()	→ 0.0
float(3)	→ 3.0
float("3.14")	→ 3.14
float(" 3.14 ")	→ 3.14
float("inf")	→ inf
float("-inf")	→ -inf
float("nan")	→ nan
float("1e-10")	→ 1e-10
float(True)	→ 1.0
float(False)	→ 0.0
float("3,14")	→ ValueError (virgule non gérée)
complex() :	
complex()	→ 0j
complex(3)	→ (3+0j)
complex(3, 4)	→ (3+4j)
complex("3+4j")	→ (3+4j)
complex("3+4J")	→ (3+4j)
complex(real=3, imag=4)	→ (3+4j)
bool() :	
bool()	→ False

```

bool(0)                → False
bool(1)                → True
bool(-1)               → True   (tout non-nul est True)
bool("")               → False
bool("hello")          → True
bool([])               → False
bool([0])              → True   (liste non-vide)
bool(None)             → False
bool(0.0)              → False
bool(0j)               → False
# VALEURS FAUSSES (falsy) : 0, 0.0, 0j, "", [], {}, set(), None, False
# TOUT LE RESTE est truthy

abs() :
abs(-5)                → 5
abs(3.14)               → 3.14
abs(-3.14)              → 3.14
abs(3+4j)               → 5.0   (module du complexe)
abs(True)               → 1
# abs() appelle __abs__() – peut être surchargé sur des classes custom

round() :
round(3.7)              → 4
round(2.5)              → 2     (banker's rounding : vers le pair !)
round(3.5)              → 4     (banker's rounding)
round(-0.5)             → 0
round(3.14159, 2)       → 3.14
round(3.14159, 0)       → 3.0   (float, pas int)
round(314, -2)           → 300   (arrondi à la centaine)
round(1234, -3)         → 1000
round(1500, -3)         → 2000   (banker's rounding)
# round() appelle __round__() – fonctionne sur Decimal, Fraction...
round(Decimal("2.355"), 2) → Decimal('2.36')

divmod() :
divmod(7, 2)            → (3, 1) (quotient, reste)
divmod(-7, 2)           → (-4, 1) (floor division + modulo)
divmod(7.5, 2.5)        → (3.0, 0.0)
divmod(7, -2)           → (-4, -1)
q, r = divmod(n, d)
# Équivalent à : (n // d, n % d)

pow() :
pow(2, 10)              → 1024
pow(2, -1)              → 0.5   (float si exposant négatif)
pow(2, 0.5)             → 1.4142... (float)
pow(2, 10, 1000)        → 24    (modulo : 2^10 mod 1000)
pow(2, -1, 3)           → 2     (inverse modulaire !)
# pow(x, y, mod) est plus efficace que (x**y) % mod pour grands entiers
# pow(base, -1, mod) = inverse modulaire (si base et mod sont premiers entre eux)

```

Conversions chaînes et bytes

DESCRIPTION : Conversion vers et depuis les types texte et binaire.
POURQUOI : Sérialisation, encodage, affichage, protocoles binaires.

```

str() :
str()                  → ""
str(42)                → "42"
str(3.14)              → "3.14"
str(True)              → "True"
str(None)              → "None"
str([1, 2, 3])         → "[1, 2, 3]"
str(b"hello")          → "b'hello'" (pas décodé !)
str(b"hello", "utf-8") → "hello"   (décodage)
str(b"hello", encoding="utf-8") → "hello"
str(b"\xff", "utf-8", errors="replace") → "?"
# str(obj) appelle obj.__str__()
# Si __str__ n'est pas défini, utilise __repr__

repr() :
repr(42)               → "42"
repr("hello")          → "'hello'"
repr([1, 2, 3])        → "[1, 2, 3]"

```

```

repr(None)                → "None"
repr(b"hello")            → "b'hello'"
repr(3+4j)                → "(3+4j)"
# repr() retourne une chaîne qui peut être réévaluée par eval()
# repr() appelle obj.__repr__()

chr() :
chr(65)                   → "A"
chr(97)                   → "a"
chr(48)                   → "0"
chr(0x1F600)              → "𐀀"
chr(0)                    → "\x00"
chr(1114111)              → "\U0010ffff" (max Unicode)
chr(1114112)              → ValueError

ord() :
ord("A")                  → 65
ord("a")                  → 97
ord("0")                  → 48
ord("𐀀")                  → 128512
ord("AB")                  → TypeError (str de longueur > 1)
# chr() et ord() sont inverses

format() :
format(3.14159, ".2f")    → "3.14"
format(1234567, ",")      → "1,234,567"
format(0.75, "%")         → "75.000000%"
format(255, "x")          → "ff" (hexadécimal)
format(255, "X")          → "FF"
format(255, "b")          → "11111111" (binaire)
format(255, "o")          → "377" (octal)
format(255, "#x")         → "0xff"
format(3.14, "e")         → "3.140000e+00"
format(3.14, ".2e")       → "3.14e+00"
format(42, "10d")         → "42" (largeur 10)
format(42, "<10d")         → "42" (gauche)
format(42, "^10d")        → " 42 " (centré)
format(42, "0>10d")       → "0000000042" (rempli de 0)
format(obj, spec)         → appelle obj.__format__(spec)

bytes() :
bytes()                   → b""
bytes(5)                  → b"\x00\x00\x00\x00\x00"
bytes([72, 101, 108, 108, 111]) → b"Hello"
bytes("hello", "utf-8")    → b"hello"
bytes("hello", encoding="utf-8") → b"hello"
bytes(range(5))            → b"\x00\x01\x02\x03\x04"
b"hello"                  Littéral bytes

bytearray() :
bytearray()               → bytearray(b"")
bytearray(5)              → bytearray(b"\x00\x00\x00\x00\x00")
bytearray([65, 66, 67])   → bytearray(b"ABC")
bytearray("hello", "utf-8") → bytearray(b"hello")
ba = bytearray(b"hello")
ba[0] = 72                → bytearray(b"Hello") (mutable !)
# bytes : immuable, bytearray : mutable
bytearray.fromhex("48656c6c66") → bytearray(b"Hello")
bytes.fromhex("48656c6c66")    → b"Hello"

memoryview() :
mv = memoryview(b"hello")    Vue sans copie
mv[0]                        → 104 (h)
mv[1:3]                      → <memory...> (slice sans copie)
mv.tobytes()                 → b"hello"
mv.tolist()                  → [104, 101, 108, 108, 111]
mv.hex()                     → "68656c6c66"
mv.itemsize                  → 1
mv.format                    → "B"
mv.shape                     → (5,)
mv.ndim                      → 1
# Utile pour buffer protocol – évite les copies inutiles

bin() / hex() / oct() :
bin(10)                      → "0b1010"

```

```

bin(-10)                → "-0b1010"
hex(255)                 → "0xff"
hex(-255)                → "-0xff"
oct(8)                   → "0o10"
oct(255)                 → "0o377"
# Inverses : int("0b1010", 2), int("0xff", 16), int("0o17", 8)

```

Types conteneurs

DESCRIPTION : Création et conversion des types conteneurs de base.
POURQUOI : Les conteneurs sont la base de la manipulation de données Python.

```

list() :
    list()                → []
    list("abc")           → ["a", "b", "c"]
    list([1, 2, 3])       → [1, 2, 3]
    list({1, 2, 3})       → [1, 2, 3] (ordre non garanti)
    list(range(5))        → [0, 1, 2, 3, 4]
    list(d.items())       → [(k,v), ...]
    list(map(f, it))      → résultats de map comme liste

tuple() :
    tuple()               → ()
    tuple("abc")          → ("a", "b", "c")
    tuple([1, 2, 3])      → (1, 2, 3)
    tuple(range(5))       → (0, 1, 2, 3, 4)

dict() :
    dict()                → {}
    dict(a=1, b=2)        → {"a": 1, "b": 2}
    dict([("a",1), ("b",2)]) → {"a": 1, "b": 2}
    dict(zip(clés, valeurs)) → dict depuis deux listes
    dict(**d1, **d2)      → Fusion (Python 3.9+ : d1 | d2)
    {k: v for k, v in items} → Dict comprehension

set() :
    set()                 → set() (pas {} qui est un dict vide !)
    set("abcabc")         → {"a", "b", "c"}
    set([1, 2, 2, 3])     → {1, 2, 3}
    set(range(5))         → {0, 1, 2, 3, 4}

frozenset() :
    frozenset()           → frozenset()
    frozenset([1, 2, 3])  → frozenset({1, 2, 3})
    frozenset("abc")      → frozenset({"a","b","c"})
    # frozenset est immuable → hashable → peut être clé de dict ou élément de set

```

Fonctions numériques

DESCRIPTION : Opérations numériques fondamentales sur iterables.
POURQUOI : Calculs agrégés rapides, en une ligne.

```

sum() :
    sum([1, 2, 3, 4])    → 10
    sum(range(101))      → 5050
    sum([1, 2, 3], start=10) → 16 (start = valeur initiale)
    sum([[1,2],[3,4]], start=[]) → [1, 2, 3, 4] (concatène listes)
    sum(x**2 for x in range(5)) → 30 (generator expression)
    # sum() accepte tout itérable de valeurs additionnables
    # Ne pas utiliser sum() sur des chaînes → utiliser "".join()

min() :
    min(3, 1, 4, 1, 5)   → 1 (plusieurs arguments)
    min([3, 1, 4, 1, 5]) → 1 (un itérable)
    min("abc", "aaa")     → "aaa" (ordre lexicographique)
    min([])               → ValueError (itérable vide)
    min([], default=0)    → 0 (valeur par défaut si vide)
    min([3,1,4], key=abs) → 1 (clé personnalisée)
    min(["banana","apple","pear"], key=len) → "pear" (plus court)
    min(data, key=lambda x: x["age"]) → élément avec age minimal

max() :
    max(3, 1, 4, 1, 5)   → 5

```

```

max([3, 1, 4, 1, 5])           → 5
max([], default=-1)           → -1
max(data, key=lambda x: x["score"])

len() :
len([1, 2, 3])                 → 3
len("hello")                   → 5
len((1, 2, 3))                 → 3
len({1, 2, 3})                 → 3
len({"a": 1, "b": 2})          → 2
len(range(10, 20))             → 10
len(b"hello")                  → 5
# len() appelle __len__()
# Ne fonctionne pas sur les générateurs (pas de longueur définie)

```

Itérables et séquences

DESCRIPTION : Fonctions pour créer, transformer et parcourir des itérables.
POURQUOI : Traitement fonctionnel des collections, Python idiomatique.
CONTEXTE : Boucles, transformations, filtrage, tri.

```

range() :
range(n)                0, 1, ..., n-1
range(5)                 → 0, 1, 2, 3, 4
range(start, stop)       start, start+1, ..., stop-1
range(1, 6)              → 1, 2, 3, 4, 5
range(start, stop, step) avec pas
range(0, 10, 2)          → 0, 2, 4, 6, 8
range(10, 0, -1)         → 10, 9, 8, ..., 1
range(10, -1, -1)        → 10, 9, ..., 0
range(0, 10, 3)          → 0, 3, 6, 9
list(range(5))            → [0, 1, 2, 3, 4]
range(0, 10, 2)[3]       → 6 (indexable, 0(1))
len(range(0, 10, 2))     → 5
5 in range(0, 10, 2)     → False (0(1))
range(5) == range(0, 5, 1) → True
# range retourne un objet lazy (ne génère pas tous les entiers en mémoire)

enumerate() :
enumerate(iterable)      (index, valeur) à partir de 0
enumerate(["a","b","c"]) → (0,"a"), (1,"b"), (2,"c")
enumerate(["a","b","c"], start=1) → (1,"a"), (2,"b"), (3,"c")
for i, val in enumerate(liste): Idiom Python pour index + valeur
list(enumerate("abc"))     → [(0,"a"), (1,"b"), (2,"c")]
# Préférer enumerate() à range(len(lst)) + lst[i]

```

```

zip() :
zip(a, b)                Paire d'éléments correspondants
zip([1,2,3], ["a","b","c"]) → (1,"a"), (2,"b"), (3,"c")
zip([1,2,3], ["a","b"])   → (1,"a"), (2,"b") (s'arrête au plus court)
list(zip(range(3), "abc")) → [(0,"a"), (1,"b"), (2,"c")]
for a, b in zip(liste1, liste2): Itère deux listes en parallèle
dict(zip(clés, valeurs))   Crée un dict depuis deux listes
zip(*matrice)              Transposition de matrice
a, b, c = zip(*(1,4),(2,5),(3,6)) Dépaquetage transposé
from itertools import zip_longest
zip_longest(a, b, fillvalue=None) Complète avec fillvalue

```

```

map() :
map(fonction, iterable)   Applique fonction à chaque élément
map(str, [1, 2, 3])       → "1", "2", "3"
map(abs, [-1, -2, 3])     → 1, 2, 3
map(int, ["1", "2", "3"]) → 1, 2, 3
map(lambda x: x**2, range(5)) → 0, 1, 4, 9, 16
list(map(f, it))           Convertit en liste
map(fonction, it1, it2)    Deux itérables → fonction(a, b)
map(pow, [2,3,4], [10,2,3]) → 1024, 9, 64
# map() retourne un iterator lazy
# Préférer les list comprehensions pour la lisibilité
# [f(x) for x in it] est équivalent et souvent plus lisible

```

```

filter() :
filter(fonction, iterable) Garde les éléments où fonction(x) est True
filter(None, [0, 1, "", "a", [], [1]]) → 1, "a", [1] (None → filtre les falsy)

```

```

filter(lambda x: x > 0, [-1, 0, 1, 2])      → 1, 2
list(filter(str.isdigit, ["1","a","2"]))    → ["1", "2"]
# filter() retourne un iterator lazy
# [x for x in it if condition(x)] est souvent plus lisible

sorted() :
sorted(iterable)                           Retourne une nouvelle liste triée
sorted([3, 1, 4, 1, 5])                    → [1, 1, 3, 4, 5]
sorted("hello")                             → ["e", "h", "l", "l", "o"]
sorted([3, 1, 4], reverse=True)             → [4, 3, 1]
sorted(data, key=lambda x: x["age"])        Tri par clé
sorted(data, key=lambda x: (x["nom"], x["age"])) Tri multi-critères
sorted(mots, key=str.lower)                 Tri insensible à la casse
sorted(mots, key=len)                      Tri par longueur
from functools import cmp_to_key
sorted(data, key=cmp_to_key(comparateur))    Depuis une fonction de comparaison
# sorted() est stable : éléments égaux gardent leur ordre relatif
# Algorithme : Timsort (O(n log n))
# Différence : sorted() retourne une nouvelle liste, list.sort() modifie en place

reversed() :
reversed([1, 2, 3])                        → 3, 2, 1 (iterator)
list(reversed([1, 2, 3]))                  → [3, 2, 1]
reversed("hello")                          → TypeError (str non reversible directement)
list(reversed(range(5)))                   → [4, 3, 2, 1, 0]
# Fonctionne sur les séquences (__reversed__ ou __len__+__getitem__)
# Pour renverser une chaîne : "hello"[::-1]

iter() :
iter(iterable)                             Crée un iterator depuis un iterable
iter([1, 2, 3])                            → <list_iterator>
iter(fonction, sentinel)                   Appelle fonction jusqu'à sentinel
iter(lambda: f.read(1), "")                 Lit octet par octet jusqu'à ""
# iter(f, s) appelle f() répétitivement jusqu'à f() == s

next() :
next(iterator)                             Prochain élément
next(iterator, default)                    Prochain, ou default si épuisé
it = iter([1, 2, 3])
next(it)                                   → 1
next(it)                                   → 2
next(it)                                   → 3
next(it)                                   → StopIteration
next(it, None)                             → None (pas d'exception)
next(iter([]), "vide")                     → "vide"

all() :
all(iterable)                              True si TOUS les éléments sont truthy
all([True, True, True])                    → True
all([True, False, True])                   → False
all([])                                     → True (vide = True !)
all(x > 0 for x in [1, 2, 3])               → True
all(x > 0 for x in [1, -1, 3])              → False
all(isinstance(x, int) for x in liste)     Vérifie le type de chaque élément
# Court-circuit : s'arrête au premier False

any() :
any(iterable)                              True si AU MOINS UN élément est truthy
any([False, False, True])                  → True
any([False, False, False])                 → False
any([])                                     → False (vide = False !)
any(x > 0 for x in [-1, -2, 3])             → True
any(x > 0 for x in [-1, -2, -3])            → False
# Court-circuit : s'arrête au premier True

```

Inspection et réflexion

DESCRIPTION : Inspecter les objets, types, attributs et la hiérarchie de classes.
POURQUOI : Débogage, métaprogrammation, code générique.
CONTEXTE : Frameworks, ORMs, tests, débogage dynamique.

```

type() :
type(42)                                → <class 'int'>
type("hello")                           → <class 'str'>

```

```

type([])                → <class 'list'>
type(None)              → <class 'NoneType'>
type(lambda: None)      → <class 'function'>
type(42) == int         → True
type(42) is int         → True (préférer isinstance())
type(True) is bool      → True
type(True) is int       → False (bool est sous-classe de int)
# Créer une classe dynamiquement :
MaClasse = type("MaClasse", (object,), {"attr": 42, "methode": lambda self: None})

isinstance() :
    isinstance(42, int)        → True
    isinstance(True, int)      → True (bool est sous-classe de int !)
    isinstance(True, bool)     → True
    isinstance(3.14, (int, float)) → True (tuple de types)
    isinstance("hello", str)   → True
    isinstance([], list)       → True
    isinstance([], (list, tuple)) → True
    isinstance(None, type(None)) → True
    # Préférer isinstance() à type() == pour gérer l'héritage

issubclass() :
    issubclass(bool, int)      → True
    issubclass(int, object)    → True (tout hérite d'object)
    issubclass(list, (list, tuple)) → True (tuple de classes)
    issubclass(str, str)       → True (une classe est sous-classe d'elle-même)
    issubclass(int, str)       → False

id() :
    id(42)                    → adresse mémoire (entier)
    id("hello")               → adresse mémoire
    a = [1, 2, 3]; b = a
    id(a) == id(b)            → True (même objet)
    id(a) == id([1, 2, 3])    → False (objets différents)
    # id() est l'identité de l'objet (adresse en CPython)
    # a is b ⇔ id(a) == id(b)

hash() :
    hash(42)                  → 42 (les ints hachent à eux-mêmes)
    hash("hello")             → entier (varie selon le processus)
    hash((1, 2, 3))           → entier (tuple hashable)
    hash([1, 2, 3])           → TypeError (list non hashable)
    hash(None)                → entier
    hash(True)                → 1
    hash(False)               → 0
    # hash(1) == hash(1.0) == hash(True) → True (mêmes valeurs = même hash)
    # Un objet hashable peut être clé de dict ou élément de set

dir() :
    dir()                     → noms dans le scope local
    dir(obj)                  → liste des attributs et méthodes
    dir([])                   → ["append", "clear", "copy", ...]
    dir(str)                  → méthodes de str
    dir(module)               → noms exportés par le module
    # dir() appelle __dir__() – liste peut être surchargée
    # Utile en mode interactif pour explorer un objet

vars() :
    vars()                    → __dict__ du scope local (= locals())
    vars(obj)                 → __dict__ de obj
    vars(module)              → namespace du module
    class Foo: x = 1
    vars(Foo)                 → mappingproxy({"x": 1, ...})
    # vars() sans argument est équivalent à locals()

repr() :
    repr(42)                  → "42"
    repr("hello")             → "'hello'"
    repr([1, "a", None])      → "[1, 'a', None]"
    repr(1+2j)                → "(1+2j)"
    # repr(obj) appelle obj.__repr__()
    # Doit retourner une chaîne qui peut recréer l'objet si possible : eval(repr(x)) == x

callable() :
    callable(print)           → True

```



```

callable(len)           → True
callable(42)            → False
callable(lambda: None)  → True
callable([])            → False
callable(int)           → True   (classe = callable)
# callable(obj) vérifie si obj.__call__ existe

```

Attributs et objets

DESCRIPTION : Accès, modification et test des attributs d'objets.
 POURQUOI : Programmation générique, métaprogrammation, frameworks.

```

getattr() :
  getattr(obj, "nom")           Équivalent à obj.nom
  getattr(obj, "nom", default)  Retourne default si attribut absent
  getattr([], "append")        → <built-in method append>
  getattr([], "inexistant")     → AttributeError
  getattr([], "inexistant", None) → None
  # Utile quand le nom de l'attribut est dynamique (chaîne calculée)
  méthode = getattr(obj, méthode_name)
  méthode()                     Appel dynamique

```

```

setattr() :
  setattr(obj, "nom", valeur)   Équivalent à obj.nom = valeur
  setattr(obj, "x", 42)        obj.x = 42
  setattr(obj, attribut_name, val) Nom dynamique

```

```

delattr() :
  delattr(obj, "nom")           Équivalent à del obj.nom
  delattr(obj, "x")            Supprime l'attribut x

```

```

hasattr() :
  hasattr(obj, "nom")           True si obj.nom existe
  hasattr([], "append")        → True
  hasattr([], "inexistant")    → False
  # hasattr(obj, name) est équivalent à :
  # try: getattr(obj, name); return True
  # except AttributeError: return False

```

```

object() :
  object()                     Crée une instance de base
  # Toutes les classes héritent implicitement d'object
  class MaClasse(object): pass  Explicite (inutile en Python 3)
  class MaClasse: pass          Identique en Python 3
  # object fournit : __hash__, __eq__, __str__, __repr__, __class__...

```

```

super() :
  super()                      Classe parente (dans une méthode)
  super().__init__()           Appelle __init__ du parent
  super(MaClasse, self).__init__() Forme explicite (Python 2 style)
  super().methode()            Méthode du parent

```

```

property() :
  class Cercle:
    def __init__(self, r): self._r = r
    @property
    def rayon(self): return self._r
    @rayon.setter
    def rayon(self, val):
      if val < 0: raise ValueError
      self._r = val
    @rayon.deleter
    def rayon(self): del self._r
  # Ou : rayon = property(getter, setter, deleter, doc)

```

```

classmethod() :
  class Foo:
    @classmethod
    def creer(cls, val): return cls(val)
  # cls = la classe (pas l'instance)
  Foo.creer(42)                Appel via la classe

```

```

staticmethod() :
  class Foo:

```

```

    @staticmethod
    def utilitaire(x): return x * 2
# Pas de self ni cls – simple fonction dans le namespace de la classe

```

Évaluation et compilation

DESCRIPTION : Évaluation et exécution dynamique de code Python.
 POURQUOI : Interpréteurs, DSLs, configuration dynamique.
 CONTEXTE : Outils de développement, systèmes de plugin, configuration.
 AVERTISSEMENT : Risque de sécurité – ne jamais eval() une entrée utilisateur non filtrée.

```

eval() :
    eval("1 + 2")                → 3
    eval("x * 2", {"x": 5})      → 10 (globals explicites)
    eval("x + y", {"x": 1}, {"y": 2}) → 3 (globals, locals)
    eval("[x**2 for x in range(5)]") → [0, 1, 4, 9, 16]
    eval(compile("1+2", "", "eval")) Depuis code compilé
# eval() exécute UNE expression et retourne sa valeur
# ⚠ DANGER : eval(input()) peut exécuter du code arbitraire

exec() :
    exec("x = 1 + 2")            Exécute des instructions (pas de retour)
    exec("print('hello')")      → hello
    exec("def f(): return 42", globals_dict)
    exec("""
    for i in range(3):
        print(i)
    """)
# exec() retourne toujours None
# ⚠ DANGER : même risque qu'eval()

compile() :
    code = compile("1 + 2", "<string>", "eval") → code object
    eval(code)                                → 3
    code = compile("x = 1", "<string>", "exec")
    exec(code)
    compile(source, filename, mode)
# mode : "eval" (expression), "exec" (instructions), "single" (REPL)
# Utile pour compiler une fois et exécuter plusieurs fois

__import__() :
    math = __import__("math")                Import dynamique (bas niveau)
# Préférer :
    import importlib
    importlib.import_module("math")           Import dynamique (haut niveau)
    importlib.import_module(".utils", "mypackage") Import relatif

```

slice() et autres

DESCRIPTION : Objet slice, aide, assertion, impression.

```

slice() :
    s = slice(5)                    → slice(None, 5, None) = [:5]
    s = slice(1, 5)                → slice(1, 5, None) = [1:5]
    s = slice(1, 10, 2)            → slice(1, 10, 2) = [1:10:2]
    liste[slice(1, 5)]             → liste[1:5]
    s.start, s.stop, s.step        Attributs
    s.indices(len(lst))            → (start, stop, step) normalisés
# Utile dans __getitem__ pour les classes custom
def __getitem__(self, key):
    if isinstance(key, slice): ...

help() :
    help()                         Démarrage de l'aide interactive
    help(len)                      Documentation de len
    help(list)                     Documentation de list
    help(list.append)              Documentation de append
    help("topics")                 Topics disponibles

globals() :
    globals()                      Dict du namespace global actuel
    globals()["x"] = 42             Équivalent à créer x = 42 globalement

```

```

locals() :
    locals()                Dict du namespace local actuel (copie)
    # Dans une fonction : locals() capture les variables locales
    # Modifier locals() n'affecte pas les variables locales réelles

zip() (avancé) :
    zip(*[[1,4],[2,5],[3,6]])      → (1,2,3), (4,5,6) (transposition)
    keys = ["a","b","c"]; vals = [1,2,3]
    dict(zip(keys, vals))           → {"a":1, "b":2, "c":3}

```

Constantes intégrées

DESCRIPTION : Constantes et exceptions disponibles sans import.

CONSTANTES :

None	Valeur nulle
True	Booléen vrai
False	Booléen faux
...	Ellipsis (Ellipsis)
__name__	Nom du module courant
__file__	Chemin du fichier courant
__doc__	Docstring du module courant
NotImplemented	Retourné par __eq__ etc. si pas implémenté

EXCEPTIONS INTÉGRÉES (les plus courantes) :

Exception	Base de toutes les exceptions
BaseException	Base absolue (inclut SystemExit, etc.)
ValueError	Valeur incorrecte
TypeError	Type incorrect
KeyError	Clé absente du dict
IndexError	Indice hors limites
AttributeError	Attribut inexistant
NameError	Nom non défini
FileNotFoundError	Fichier introuvable
IOError	Alias de OSError
OSError	Erreur système
PermissionError	Permission refusée (sous-classe OSError)
RuntimeError	Erreur générique d'exécution
StopIteration	Fin d'un itérateur
StopAsyncIteration	Fin d'un itérateur asynchrone
GeneratorExit	Fermeture d'un générateur
ArithmeticError	Erreur arithmétique (base)
ZeroDivisionError	Division par zéro
OverflowError	Dépassement numérique
FloatingPointError	Erreur flottante
ImportError	Échec d'import
ModuleNotFoundError	Module introuvable (sous-classe ImportError)
NotImplementedError	Méthode non implémentée
RecursionError	Récursion trop profonde
MemoryError	Mémoire insuffisante
UnicodeError	Erreur Unicode (base)
UnicodeDecodeError	Décodage impossible
UnicodeEncodeError	Encodage impossible
AssertionError	assert a échoué
EOFError	Fin de fichier inattendue
KeyboardInterrupt	Ctrl+C
SystemExit	sys.exit() appelé
SyntaxError	Erreur de syntaxe
IndentationError	Erreur d'indentation
LookupError	Base de KeyError et IndexError
ConnectionError	Erreur réseau (base)
TimeoutError	Délai dépassé
NotADirectoryError	Pas un répertoire
IsADirectoryError	Est un répertoire
ChildProcessError	Erreur processus enfant
BrokenPipeError	Pipe brisé
BlockingIOError	Opération bloquante

AVERTISSEMENTS :

Warning	Base des avertissements
DeprecationWarning	Fonctionnalité dépréciée
UserWarning	Avertissement utilisateur
FutureWarning	Changement de comportement futur
RuntimeWarning	Avertissement d'exécution

RÉCAPITULATIF COMPLET DES BUILTINS

— ENTRÉE / SORTIE —

```
print(*objects, sep=" ", end="\n", file=stdout, flush=False)
input(prompt="")
open(file, mode="r", buffering=-1, encoding=None, errors=None, newline=None)
```

— CONVERSIONS NUMÉRIQUES —

```
int(x, base=10)    float(x)    complex(real, imag)    bool(x)
abs(x)            round(x, n)    divmod(a, b)           pow(x, y, mod)
```

— CONVERSIONS TEXTE / BINAIRE —

```
str(obj)          repr(obj)    chr(i)                ord(c)
format(obj, spec) bin(x)        hex(x)                oct(x)
bytes(...)        bytearray(...)    memoryview(obj)
```

— TYPES CONTENEURS —

```
list(it)          tuple(it)    dict(...)    set(it)    frozenset(it)
```

— NUMÉRIQUES —

```
sum(it, start=0)    min(it, key, default)    max(it, key, default)    len(obj)
```

— ITÉRABLES —

```
range(start, stop, step)    enumerate(it, start=0)
zip(*its)                  map(f, *its)
filter(f, it)              sorted(it, key, reverse)
reversed(seq)              iter(it) / iter(f, sentinel)
next(it, default)         all(it)                any(it)
```

— INSPECTION —

```
type(obj) / type(name, bases, dict)
isinstance(obj, type_ou_tuple)
issubclass(cls, classe_ou_tuple)
id(obj)    hash(obj)    dir(obj)    vars(obj)
repr(obj)    callable(obj)
```

— ATTRIBUTS —

```
getattr(obj, name, default)    setattr(obj, name, val)
delattr(obj, name)             hasattr(obj, name)
```

— CLASSES —

```
object()    super()    property()
classmethod()    staticmethod()
```

— ÉVALUATION —

```
eval(expr, globals, locals)    exec(code, globals, locals)
compile(source, file, mode)    __import__(name)
```

— DIVERS —

```
slice(start, stop, step)    help(obj)
globals()                  locals()
```

— GUIDE DE CHOIX RAPIDE —

BESOIN	BUILTIN(S)
--------	------------

Index + valeur dans une boucle	enumerate()
Deux listes en parallèle	zip()
Transformer chaque élément	[f(x) for x in it] ou map()
Filtrer des éléments	[x for x in it if c] ou filter()
Tous les éléments vérifient cond	all(cond(x) for x in it)
Au moins un vérifie cond	any(cond(x) for x in it)
Trier avec critère custom	sorted(it, key=lambda x: ...)
Chercher min/max avec critère	min/max(it, key=lambda x: ...)
Somme de carrés	sum(x**2 for x in it)
Produit (Python 3.8+)	math.prod(it)
Dict depuis deux listes	dict(zip(clés, valeurs))
Transposer une matrice	list(zip(*matrice))
Vérifier le type	isinstance(x, (int, float))
Attribut dynamique	getattr(obj, nom, défaut)
Appel dynamique	getattr(obj, "méthode")()
Lire un fichier entier	open("f").read()

Lire ligne par ligne for ligne in open("f"):

— PIÈGES COURANTS —

set() → set vide OK MAIS {} → dict vide (pas un set !)
bool([0]) → True (liste non-vide même si contient seulement 0)
bool(0.0) → False
round(2.5) → 2 (banker's rounding, pas 3 !)
round(3.5) → 4
int(-3.7) → -3 (troncature, pas floor)
math.floor(-3.7) → -4 (plancher)
type(True) is int → False (True est bool, sous-classe de int)
isinstance(True, int) → True
all([]) → True (vide = vacuously true)
any([]) → False (vide = aucun truthy)
reversed("hello") → TypeError (utiliser "hello"[::-1])
zip arrête au plus court → utiliser itertools.zip_longest si nécessaire
sorted() retourne une liste, list.sort() modifie en place et retourne None
map() et filter() retournent des iterators lazy (pas des listes)
str(b"hello") → "b'hello'" (pas "hello") – utiliser b"hello".decode()
locals() → copie – modifier locals() n'affecte pas les vraies variables
eval(input()) → DANGER absolu – ne jamais faire

=====